

Code and data for "Prediction-Driven Non-Preemptive Scheduling with Bounded Overtaking: A Per-Job Wait Guarantee"

Yazhou Guo, Zexin Lin, Chengyang Huo, Yurong Song. Manuscript mathematics-4625685, under review at *Mathematics* (MDPI), 2026.

Version 1.0.1-submitted. Archived at Zenodo, <https://doi.org/10.5281/zenodo.23039931> (all versions: <https://doi.org/10.5281/zenodo.23035899>); the same files are in the GitHub repository <https://github.com/oscar030406/spjf-guard> under the tag v1.0.1-submitted. Version 1.0.0-submitted (doi:10.5281/zenodo.23035900) has the same code, data and results under the manuscript's earlier title.

This package holds the code, the configuration, the result tables and the run logs behind every number in the manuscript and its Supplementary Materials, in the version that was submitted. It rebuilds those numbers from the public datasets the paper uses. It does not contain the datasets themselves (see "Data availability") or the manuscript.

The layout follows the replication-package template of the Social Science Data Editors (the AEA template README, v1.1): data availability, computational requirements, a description of the programs, instructions, and a list of tables with the programs and files that produce them.

Contents

Path	What it holds
README.md, README.pdf	this file
src/spjf_guard/	the Python package that runs the main experiment: the queue simulator in front of k servers, which checks the paper's per-job bound on every simulated job (<code>sim/</code>); dataset readers and the lock on the sealed data (<code>data/</code>); causal features (<code>features/</code>); the running-time predictor (<code>predict/</code>); replay traces, parameter selection, metrics and tables (<code>experiment/</code>)
scripts/	the command-line steps of the pipeline, listed under "Instructions to replicators"
configs/	<code>main.yaml</code> holds every setting of the main experiment, each with a comment; the dated files are the configurations of the sensitivity runs and of the visibility runs
tests/	unit and property tests of the package
evidence/	the separate studies behind the numbers the package does not produce (other workloads, the predictor comparison, numerical checks of the propositions, capacity and physical runs), one folder per study with its scripts, its logs (<code>out_*.txt</code>) and a <code>README.md</code> ; <code>evidence/README.md</code> lists them
outputs/	the result tables the submitted manuscript reports, as the package wrote them; <code>outputs/README.md</code> says which command wrote each directory
data/	empty apart from <code>data/README.md</code> : where to download each dataset, where to save it, and the SHA-256 of the files we used
docs/	<code>adr/</code> : design decisions with the alternatives rejected; <code>sealed_run_procedure.md</code> : how the sealed semesters were run once; <code>sealed_access_log.md</code> (with an English rendering, <code>sealed_access_log.en.md</code>): every read of sealed data; <code>post_run_changes.json</code> : reporting changes made after the sealed run
prechecks/timeout_rule/	one kernel the online replay imports from this path
<code>protocol_lock.json</code> , <code>protocol_lock.draft.json</code>	the hashes of code, configuration and inputs at the freeze of 2026-09-25, and the pinned choices (split, seeds, selected parameters)
CONTEXT.md	glossary: one term per concept, used identically in the paper, the code and the tables
GENERATED.md	every generated file: its source, the command that regenerates it and the command that checks it
<code>pyproject.toml</code> , <code>uv.lock</code>	the Python version and the locked dependency versions
CITATION.cff, LICENSE	how to cite; MIT licence for the code

Path	What it holds
MANIFEST.sha256	SHA-256 of every file in the package

Data availability and provenance

Statement about rights

The authors have legitimate access to and permission to use every dataset in this study. All of them are published by third parties and are publicly downloadable; none needs an access agreement. We redistribute none of them, as the manuscript's Data Availability Statement says. What the package contains is code, configuration, aggregate tables and run logs: no dataset row, student identifier, user name or line of student source code.

Datasets

Dataset	Provided here	Where to get it	Terms
CodeBench v1.81 (Universidade Federal do Amazonas), 18 semesters	no	https://codebench.icomp.ufam.edu.br/dataset/	no licence stated; cited, not redistributed
ACcoding v1.0.0 (Chen et al. 2024, doi:10.1038/s41597-024-03392-z)	no	doi:10.5281/zenodo.6522395	CC BY 4.0 per the data descriptor
OULAD (Kuzilek et al. 2017, doi:10.1038/sdata.2017.171)	no	https://analyse.kmi.open.ac.uk/open_dataset	CC BY 4.0
Azure Functions Invocation Trace 2021 (Zhang et al., SOSP 2021)	no	https://github.com/Azure/AzurePublicDataset	CC BY 4.0
Intel Netbatch 2012, Parallel Workloads Archive (Shai, Shmueli and Feitelson, JSSPP 2013)	no	https://www.cs.huji.ac.il/labs/parallel/workload/I_intel_netbatch/	free for research, with acknowledgement
LPC-EGEE 2004, Parallel Workloads Archive	no	https://www.cs.huji.ac.il/labs/parallel/workload/I_lpc/	free for research, with acknowledgement
Mozilla Firefox CI, two hardware worker pools, August–September 2026	no	collected from Mozilla's public Taskcluster and Treeherder APIs by <code>evidence/firefox_ci/run_collect.sh</code>	no licence stated; attributed to Mozilla

`data/README.md` gives the path each file must be saved at, the checksums of the files we used, and the acknowledgements the archives ask for.

Computational requirements

Software

Python 3.12 with the dependency versions locked in `uv.lock` (numpy, pandas, pyarrow, numba, LightGBM, and pytest, ruff and mypy for the checks), installed with [uv](#). The reported runs were made on Windows 11 x86-64. On Linux with the same locked versions, the refit of the frozen predictor differs in the last bit of the score for 714 of the 147,380 jobs of the first target semester (checked on 2026-09-25), and the runners that assert the refit equals the released scores stop there; everything else rebuilds.

Controlled randomness

Every random draw uses numpy's `default_rng` with a fixed seed:

- the copy shifts of the replay traces: `overlay.seed = 3` in `configs/main.yaml` (`seed × 100 + overlay`, `src/spjf_guard/experiment/overlay.py`), and `seed × 100 + 50` for the single-server trace;

- the predictor: `predictor.seed = 3`, with LightGBM's thread count fixed at 4, because its histogram reduction order depends on the thread count;
- the paired week-block bootstrap: `bootstrap.seed = 20260919`;
- the class-term bootstrap: `BOOT_SEED = 20260924` in `scripts/run_cluster_bootstrap.py`;
- the audit samples and the timing sample: `scripts/run_online_visibility.py`, `scripts/run_consistent_visibility.py`, `selection_cost_seed` in `configs/main.yaml`.

Memory, runtime and storage

Measured on the authors' machine: Intel Core Ultra 9 275HX (24 threads), 64 GB of memory, Windows 11, with two worker processes and four threads each.

Step	Wall-clock time
Parse one CodeBench semester archive	about 11 seconds
Build the parsed cache, 11 development semesters	about 5 minutes
Fit the ranking scores (--repeat checks two runs are identical)	about 7 minutes
Build the replay traces	about 2 minutes
Choose the guard's parameters, 258 candidates × 15 validation cells	about 7.3 hours
Main run, 15 cells, 29 policies	about 48 minutes
Predictor metrics	about 11 minutes
Visibility comparison	about 35 minutes
Exact policy-specific visibility	about 4.3 hours
Online replay, 75 policy-cells	about 4.7 hours
Class-term bootstrap, 100 resamples, three worker processes	about 23 hours (first to last resample file)

The development pipeline takes about 41 hours in all, 23 of them in the class-term bootstrap. Free disk needed: about 30 GB (8 GB of raw data, 17 GB of derived traces, 3.5 GB of per-job files, and about 750 MB of temporary files per running cell). One online-replay worker process uses about 6 GB of memory.

Instructions to replicators

Run everything from the package root. The steps are in order; each one only reads what the steps before it wrote.

1. **Install.** `uv sync --extra dev`, then set the command prefix every step below uses:

```
export UV="env -u PYTHONHOME -u PYTHONPATH -u UV_INTERNAL_PYTHONHOME \
  NUMBA_NUM_THREADS=4 OMP_NUM_THREADS=4 uv run --no-sync"
```

2. **Check the installation without any data** (the minimal example): the unit and property tests build their inputs synthetically. On the authors' machine 335 pass in about 72 seconds; 8 are skipped, seven because they compare the emitted tables with the manuscript source, which is not in this package, and one because it needs a git checkout.

```
$UV python -m pytest -q -m "not slow and not crosscheck"
```

3. **Download the data** into the paths listed in `data/README.md` and compare the checksums. The ACcoding dumps then go through `evidence/accoding/parse_sql.py`.
4. **Build the inputs** from the development semesters (none of these reads a sealed one):

```
DEV=2018-1,2018-2,2019-1,2019-2,2020-ERE,2020-1,2020-2,2021-1,2021-2,2022-1,2022-2
SCORES=data/derived/package_ranking_scores/forward_scores.parquet
$UV python scripts/parse_archive.py --semesters $DEV
$UV python evidence/codebench_service/code_features.py --only $DEV
$UV python scripts/build_cache.py --pool development
$UV python scripts/fit_scores.py --repeat
$UV python scripts/build_overlays.py --pool primary --score-parquet $SCORES
```

```
$UV python scripts/build_overlays.py --pool validation --score-parquet $SCORES
$UV python scripts/build_overlays.py --single-server
```

5. **Choose the parameters** on the validation traces:

```
$UV python scripts/select_parameters.py --workers 2 --out-dir outputs/selection_v4
$UV python scripts/select_aging.py --workers 2
```

6. **Run the experiments** on the primary traces:

```
$UV python scripts/run_main.py --selection outputs/selection_v4/selected_parameters.csv \
--out-dir outputs/dev_tables --workers 2
$UV python scripts/run_main.py --prefix k1 --reps 0 --levels 0 \
--selection outputs/selection_v4/selected_parameters.csv --out-dir outputs/dev_tables/k1
$UV python scripts/eval_scores.py --pool primary
$UV python scripts/run_visibility.py --pool primary --workers 2
$UV python scripts/run_consistent_visibility.py --config configs/visibility_development_20260924.yaml \
--pool primary --workers 2 --resume \
--controls data/derived/package_ranking_scores/consistent_controls_20260924.npz
$UV python scripts/run_online_visibility.py --pool primary --workers 2 --resume --policies all
$UV python scripts/online_paired_differences.py
$UV python scripts/run_cluster_bootstrap.py --resamples 100 --workers 3
```

The controls file of `run_consistent_visibility.py` is built by the same script with `--build-controls-only`; `GENERATED.md` has that command, the two sensitivity runs (`configs/sensitivity_*_20260926.yaml`) and the restricted reselection under the online replay (`scripts/select_online.py`).

7. **The sealed semesters** (2023-1, 2023-2, 2024-1) were run once, after the method was frozen on 2026-09-25 (`protocol_lock.json`). The code refuses to open them without the `--unseal` flag and a valid lock. `docs/sealed_run_procedure.md` lists the commands of that run; `docs/sealed_access_log.md` records every read.

8. **Emit the tables and compare** with the copies in `outputs/`:

```
$UV python scripts/sealed_dev_contrast.py
$UV python scripts/emit_paper_tables.py
$UV python scripts/emit_paper_tables.py --dev-exact-dir outputs/dev_consistent_visibility \
--dev-online-dir outputs/dev_online_visibility --dev-cluster-dir outputs/cluster_bootstrap \
--out-dir outputs/consistent_paper_tables
$UV python scripts/check_overlays.py
$UV python scripts/diff_dev_tables.py
```

A rerun writes into `outputs/`; compare it with this package's copy, for example with `MANIFEST.sha256`.

The studies in `evidence/` run separately. Each folder's `README.md` gives its question, the paper items it supports, its inputs and the command that reruns it. Their scripts write absolute paths as `<repo-root>` (the package root) and `<cache-dir>` (a scratch directory); set both before rerunning one.

Reproducibility scope

- **Provided and rebuilt by the commands above:** every table and figure listed below, from the raw datasets, with the result files of the reported runs in `outputs/` and the logs of the separate studies in `evidence/` to compare against.
- **Not provided:** the raw datasets (third-party; download them as `data/README.md` says); the derived traces and per-job files the pipeline writes (about 20 GB, rebuilt by the same commands); the manuscript.
- **Sealed semesters:** the three sealed CodeBench semesters were evaluated once, under the frozen protocol. Rerunning that run reproduces those numbers; it is not a new test.
- **Platform:** the reported runs rebuild bit for bit on Windows x86-64 with the locked dependencies. On Linux, the frozen predictor's refit differs in the last bit for 714 of 147,380 jobs (see "Software" above), and the runners that assert equality stop there.

List of tables and programs

"Result file" is the file in this package the printed numbers are read from; "Program" is what writes it. Rows marked *no computation* are definitions or illustrations.

Manuscript

Item	Result file	Program
Figure 1	<i>no computation</i> (three-job example)	
Figure 2	<i>no computation</i> (diagram of the guard)	
Table 1	evidence/predictor_neural/out_evaluate_core.txt	evidence/predictor_neural/
Table 2	<i>no computation</i> (policy definitions)	
Table 3	<i>no computation</i> (notation)	
Table 4	evidence/codebench_audit/out_tail_and_counts.txt; evidence/accoding_v2/out_accoding_v2.txt; evidence/ codebench_service_v2/out_service_v2.txt; evidence/ cross_domain/out_audit_azure.txt, out_audit_netbatch.txt; evidence/firefox_ci/out_SUMMARY.txt; evidence/oulad/ out_load.txt	the studies of the same names
Table 5	outputs/consistent_paper_tables/tab_visibility.tex, from outputs/dev_online_visibility/online_comparison.csv and outputs/dev_consistent_visibility/exact_comparison.csv	scripts/emit_paper_tables.py, from scripts/run_online_visibility.py and scripts/run_consistent_visibility.py
Table 6	outputs/consistent_paper_tables/tab_visibility_sealed.tex; outputs/sealed_dev_contrast/contrast.csv	the sealed run (docs/ sealed_run_procedure.md); scripts/ sealed_dev_contrast.py
Table 7	outputs/dev_tables/main_table.csv, paired_differences.csv; outputs/dev_visibility/visibility_waits_and_lag.csv; outputs/online_paired/online_paired_differences.csv	scripts/run_main.py; scripts/ run_visibility.py; scripts/ online_paired_differences.py
Table 8	evidence/timeout_rule/timeout_intervals.csv, out_timeout_overlays.txt; outputs/online_paired/ online_clock_intervals.csv; outputs/dev_tables/ main_table.csv; outputs/dev_visibility/ visibility_comparison.csv	evidence/timeout_rule/; scripts/ online_paired_differences.py; scripts/ run_main.py; scripts/run_visibility.py
Table 9	CodeBench rows: outputs/dev_tables/main_table.csv, outputs/dev_predictor/predictor_metrics.csv; other rows: evidence/cross_domain/out_cross_domain_table.txt, evidence/accoding_v2/out_accoding_v2.txt, evidence/ firefox_ci/out_SUMMARY.txt	scripts/run_main.py, scripts/ eval_scores.py; the studies of the same names

Supplementary Materials

Item	Result file	Program
Table S1	evidence/cross_domain/out_cross_domain_table.txt; evidence/ firefox_ci/out_SUMMARY.txt	evidence/cross_domain/, evidence/ firefox_ci/
Table S2	evidence/predictor_neural/out_evaluate_core.txt; evidence/ codebench_service_v2/out_service_v2.txt; evidence/ accoding_v2/out_accoding_v2.txt	the studies of the same names
Table S3	evidence/codebench_audit/out_tail_and_counts.txt; evidence/ accoding_v2/out_accoding_v2.txt; evidence/cross_domain/ out_cross_domain_table.txt; evidence/firefox_ci/ out_SUMMARY.txt	the studies of the same names
Table S4	outputs/paper_tables/tab_setup.tex, from outputs/dev_tables/ ; trace rows from evidence/main_v3/out_main_v31.txt	scripts/emit_paper_tables.py

Item	Result file	Program
Table S5	protocol_lock.draft.json (selected_parameters, family_best), which outputs/selection_v4/ must match	scripts/select_parameters.py; scripts/make_protocol_lock.py
Table S6	outputs/paper_tables/tab_guard.tex, from outputs/dev_tables/main_table.csv, policy_parameters.csv	scripts/emit_paper_tables.py from scripts/run_main.py
Table S7	outputs/paper_tables/tab_guard.tex (Guard-age and Guard-queue rows)	the same
Table S8	outputs/paper_tables/tab_adv.tex, from outputs/dev_tables/main_table.csv	the same
Table S9	outputs/paper_tables/tab_resid.tex, from outputs/dev_tables/identity_residuals.csv	the same
Table S10	outputs/paper_tables/tab_k1.tex, from outputs/dev_tables/k1/main_table.csv	the same
Table S11	outputs/selection_aging/aging_protocol.json, selected_aging.csv	scripts/select_aging.py
Table S12	outputs/dev_visibility/visibility_comparison.csv	scripts/run_visibility.py
Table S13	outputs/dev_predictor/predictor_metrics.csv; evidence/codebench_service_v2/out_service_v2.txt	scripts/eval_scores.py; evidence/ codebench_service_v2/
Table S14	evidence/ranking_score/out_table_primary_5reps.txt	evidence/ranking_score/
Table S15	evidence/codebench_service_v2/out_service_v2.txt (SUMMARY 5)	evidence/codebench_service_v2/
Table S16	outputs/paper_tables/tab_rank.tex, from outputs/dev_tables/main_table.csv	scripts/emit_paper_tables.py from scripts/run_main.py
Table S17	outputs/selection_v3/selection_protocol.json	scripts/select_parameters.py
Table S18	outputs/consistent_paper_tables/tab_cluster.tex, from outputs/cluster_bootstrap/bootstrap_summary.csv	scripts/run_cluster_bootstrap.py, scripts/emit_paper_tables.py
Table S19	evidence/weakness1_attack/capacity_curve.csv, summary_numbers.json	evidence/weakness1_attack/
Table S20	evidence/weakness1_attack/physical_numbers.json, physical_verification.json	evidence/weakness1_attack/
Table S21	evidence/closed_loop/out/open_vs_closed_mean.csv, report_tables.md	evidence/closed_loop/make_tables.py
Table S22	evidence/lpc_egge_queues/tables.md	evidence/lpc_egge_queues/
Table S23	evidence/lpc_egge_queues/tables.md, policy_metrics.csv	evidence/lpc_egge_queues/
Table S24	evidence/lpc_egge/applicability.csv; evidence/lpc_egge_queues/tables.md	evidence/lpc_egge/, evidence/ lpc_egge_queues/
Table S25	evidence/platform_testbed/records/summary.json, simulator_replay.json	evidence/platform_testbed/
Table S26	<i>no computation</i> (comparison with the literature; evidence/guard_optimality_verify/literature_check.md)	
Tables S27–S30	outputs/consistent_paper_tables/tab_rank_sealed.tex, tab_guard_sealed.tex, tab_adv_sealed.tex, tab_k1_sealed.tex, from outputs/sealed_tables/	the sealed run; scripts/ emit_paper_tables.py
Table S31	outputs/consistent_paper_tables/tab_online_suite.tex, from outputs/dev_online_visibility/	scripts/emit_paper_tables.py from scripts/run_online_visibility.py
Table S32	outputs/online_paired/tab_s_online_pairs_body.tex	scripts/online_paired_differences.py
Table S33	outputs/online_paired/tab_s_online_clock_body.tex	scripts/online_paired_differences.py

Item	Result file	Program
Figure S1	outputs/dev_tables/main_table.csv; outputs/dev_visibility/visibility_comparison.csv; outputs/selection_v4/selected_parameters.csv; outputs/selection_aging/selected_aging.csv	scripts/emit_gap_harm_figure.py
Figure S2	outputs/paper_tables/tab_guard.tex	drawn from the table; no new computation
Figure S3	outputs/paper_tables/	drawn from the tables; no new computation

Propositions checked numerically

Paper item	Study
Section 6 and Supplementary S4, S8: the identity, the guard bound, tightness	evidence/guard_theory/
Proposition 10 (maximum waiting time) and Table 8	evidence/timeout_rule/, evidence/new_theory_refutation/
Proposition 11 (absolute bound) and Section 8.5	evidence/envelope_bound/
Proposition 12 (late completion reports)	evidence/stale_charging/
The cost ratio 150 (Sections 1, 6.4, 10)	evidence/cost_ratio/
Supplementary S7.1–S7.4	evidence/guard_optimality_verify/
Supplementary S7.6	evidence/reservation_guard/, evidence/reservation_guard_verify/

evidence/README.md maps every other sentence-level number to its study and log line.

What differs from the authors' working repository

The package is the submitted commit (4b4c871) with these changes:

- the manuscript under review (mathematics-4625685) differs from the manuscript source of that commit in wording only: the title, the abstract, the keywords, the introduction, Algorithm 1 set as a numbered listing, and one added reference. Every number the source prints appears unchanged in the manuscript under review (checked on 2026-09-29, section by section);
- left out: the manuscript sources, the Chinese copies of the documents, development tooling, and four studies that the paper does not use and no packaged script reads (evidence/README.md names them);
- added: outputs/ (the result tables, which the repository does not track), this README, data/README.md, outputs/README.md, MANIFEST.sha256, and evidence/accoding/parse_sql.py, the parser that turns the ACcoding SQL dumps into the tables evidence/accoding_v2/reads (absolute paths replaced by <repo-root>), with a short evidence/accoding/README.md;
- edited: the first line of GENERATED.md and of docs/sealed_run_procedure.md, which pointed to the Chinese originals, the accoding row of evidence/README.md, and CITATION.cff, which now also gives this version and its DOI. Nothing else in the commit was changed. The configuration files still mention data/derived/README.md, which this package does not include; they are left as they are because their hashes are recorded in the protocol lock and in every outputs/*/manifest.json.

The tables in outputs/ were checked against the submitted manuscript on 2026-09-29: with the manuscript source placed in paper/, tests/test_paper_tables.py passes and scripts/check_paper_numbers.py reports that every number the paper prints is the one the package produced.

Licence and citation

The code is released under the MIT licence (LICENSE). The datasets are not covered by it; each keeps its publisher's terms. To cite this package, cite the manuscript and this archive: Guo, Y.; Lin, Z.; Huo, C.; Song, Y. Code and data for "Prediction-Driven

Non-Preemptive Scheduling with Bounded Overtaking: A Per-Job Wait Guarantee", version 1.0.1-submitted. Zenodo, 2026, doi:10.5281/zenodo.23039931. CITATION.cff gives the same in machine-readable form.